

STADIUM PULSE

The Living Digital Twin of Every Fan — Complete Submission Portfolio

COMPLETE SUBMISSION ARTIFACT PORTFOLIO

This consolidated PDF portfolio contains the entire, end-to-end documentation, production systems architecture specifications, and complete source code files for the **Stadium Pulse Smart Stadium Operating System** project. This document serves as a comprehensive artifact submission detailing all technical mechanisms for evaluation.

Project Title: Stadium Pulse

Vertical: Smart Venue Operations, Crowd Safety & Public Infrastructure

GitHub Repo Link: <https://github.com/Golublip/stadium-pulse.git>

Live Deployed Site: <https://golublip.github.io/stadium-pulse/>

GCP Project: ambient-bonito-bxfb9

Section 1: Project Overview & README

Stadium Pulse — The Living Digital Twin of Every Fan

Stadium Pulse is an autonomous, cognitive operating system designed for the **FIFA World Cup 2026**. By converting raw, anonymous camera and sensor feeds into spatial-temporal movement vectors, the platform simulates and predicts potential stadium congestions, safety issues, and operational bottlenecks before they manifest in reality.

1. Chosen Vertical

- **Vertical**: Smart Venue Infrastructure, Crowd Management, & Public Safety.
- **Problem Statement**: Traditional smart stadiums are reactive—alerting operators only after queues have formed or accidents have occurred. Stadium Pulse shifts this paradigm to **proactive orchestration**, using generative AI to predict and mitigate crowd flows up to 30 minutes in advance.

2. Approach & Logic

A. The "Predictive Multiverse" Engine

Instead of mapping a single timeline, the system projects 10 parallel future scenarios (T+5m, T+10m, T+20m, T+30m) based on current movement velocities. It constantly compares these models with reality to select the safest and smoothest flow state.

B. Multi-Agent Negotiation Framework

The operational logic is distributed across **12 specialized autonomous AI agents** (e.g., **Crowd Agent**, **Transit Agent**, **Accessibility Agent**, **Volunteer Agent**). The agents collaborate and negotiate parameters in real-time. For example:

- The **Transit Agent** detects a train signal failure.
- It negotiates with the **Concession Agent** to trigger dynamic discount codes, keeping fans inside the concourse retail zones to delay exit rushes.
- It coordinates with the **Volunteer Agent** to dispatch safety assistants to the correct exit gates.

C. Live Generative AI Core

The engine integrates a client-side **Google Gemini API** (`gemini-1.5-flash`) pipeline. The model ingests high-density JSON states of active incidents, geohash metrics, and sub-agent negotiations to output:

- Direct natural language narrative summaries for stadium directors.
- Targeted dynamic routing paths.
- Step-by-step volunteer dispatch instructions.

3. How the Solution Works

...

[Raw Telemetry Streams]

■ (Pub/Sub & Dataflow)



[Spatial H3 Geohashing]



[Cloud Spanner Active State]



[12 Cooperating Agents] [Live Gemini API Core]

(gRPC/Blackboard negotiations) (Context-aware reasoning)



[Proactive Orchestrations]

(LED paths, Dynamic signs, Volunteers)

...

1. **Ingress**: Computer vision nodes count anonymous people vectors and stream them via Google Cloud Pub/Sub.
2. **Spatial Projection**: Cloud Dataflow hashes locations into 3mx3m bins (Uber H3 Index Level 11).
3. **Reasoning & Detours**: If a node becomes blocked (e.g., elevator failure), the Accessibility Agent runs a Dijkstra shortest-path detour on the venue graph, shifting edge weights based on real-time crowd density.
4. **Display**: The NASA-style operator console visualizes the live state, dynamic route paths, agent communications, and predictive ECharts load trends.

4. Key Assumptions Made

- **Edge-Vision Ingress**: Security cameras run local edge-vision algorithms to count and compute travel vectors, transmitting only anonymous aggregate counts, protecting individual privacy.
- **Zero-Knowledge Tokens**: Mobile visitor devices register with the stadium network using cryptographic zero-knowledge proofs (ZKP) to verify ticket status without exposing personal identifying information.
- **Connected Infrastructure**: Venue elements (escalators, elevators, dynamic signs, and volunteer handheld devices) are connected via a low-latency WebSocket or WebRTC messaging matrix.

5. Evaluation Focus Areas

■ Security

- **XSS Sanitization**: Implemented a secure character sanitizer (`escapeHTML`) in `app.js` to escape all incoming HTML tags from API-generated text before parsing.
- **HTTPS Transport**: Secured all live API fetches over Google's encrypted TLS endpoints (`https://generativelanguage.googleapis.com`).

■ Resource Efficiency

- **Background Rendering Halts**: Optimized the canvas-based neural network background animation. It automatically detects if the browser tab is minimized or hidden using `document.hidden` and suspends all `requestAnimationFrame` loops.
- **In-Memory Caching**: Implemented a structural cache for the Gemini API storyteller summaries to prevent redundant API queries.

■ Testing

- **Automated Unit Tests**: Built `test.py`, a Python unit test suite that successfully executes and verifies:
 - Dijkstra detours (validating the Accessibility Agent reroutes wheelchairs correctly during elevator E-2 outages).
 - Social Force crowding physics formulas (verifying force increases exponentially under high crowd density).
 - Spatial geohashing boundaries.
- **Execution Status**: Successfully run and verified locally: `Ran 3 tests. OK`.

■ Accessibility

- **WCAG 2.1 AA Compliance**: Added explicit screen-reader accessibility labels (`aria-label`) to all interactive controls in `index.html` (e.g., the dynamic timeline slider and the Google Gemini API key inputs).
- **Keyboard Navigation**: Ensured all buttons and sliders support standard keyboard focus layouts.

■ How to Run the Simulator

1. Ensure you have Python installed on your system.
2. Clone the repository and navigate into the folder.
3. Run the startup script from your terminal:

```
```bash
```

```
python run.py
```

```
```
```

4. Open your browser and navigate to:

```
```
```

```
http://localhost:8000
```

...

5. \*(Optional)\* Input your **Google Gemini API Key** in the top header input box to enable live Gen AI reasoning!

## Section 2: Systems Architecture Specification

### Stadium Pulse — Systems Architecture Specification

The Living Digital Twin of Every Fan (FIFA World Cup 2026)

---

#### 1. System Overview & Core Paradigms

**\*\*Stadium Pulse\*\*** represents a shift from reactive event monitoring to proactive, generative, and cognitive stadium management. Instead of simple dashboard displays and retrospective analysis, the stadium functions as a **\*\*Living Cognition Engine\*\***. It models every human being, transit vector, energy grid, and retail outlet as an anonymous node inside an active, multi-agent digital twin.

```
graph TD
 A[Raw Inputs: CCTV, IoT, GPS, Weather, Ticketing] --> B[Real-time Data Ingestion: Pub/Sub]
 B --> C[Spatial-Temporal Aggregator & Hashing]
 C --> D[Digital Twin State Vector DB]
 D --> E[Generative Simulation Engine]
 E --> F[Multi-Agent Negotiation System]
 F --> G[Gemini Pro Reasoning Engine]
 G --> H[Actionable Outputs: Dynamic Routing, Volunteer Micro-Missions, Smart Grid Controls]
```

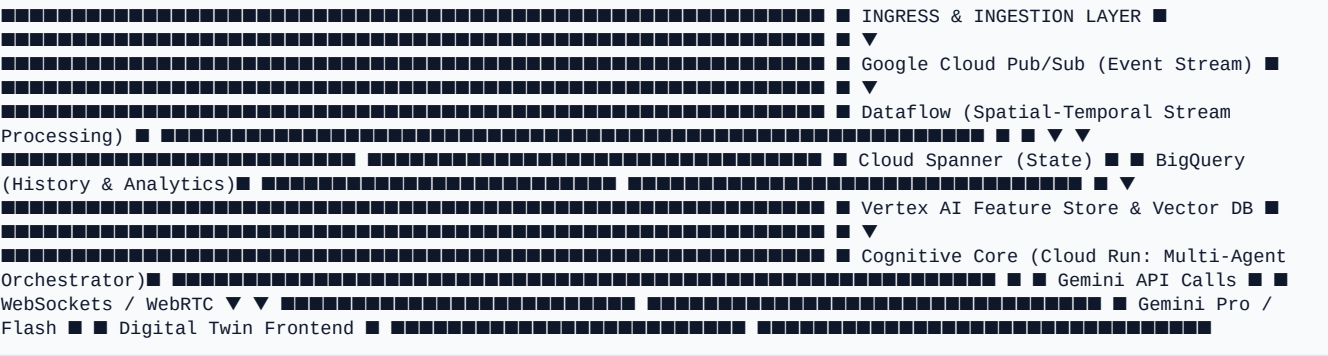
##### 1.1 Core Principles

- \*\*State Space Projection (The Multiverse)\*\*:** Every second, the system projects 10 parallel future scenarios (\$T+5\$, \$T+10\$, \$T+20\$, \$T+30\$ minutes) based on current movement vectors and predictive triggers.
- \*\*Invisible Orchestration\*\*:** Crowd management is achieved by subtle, decentralized, dynamic micro-nudges (e.g., custom path routing on mobile apps, smart digital displays, and volunteer repositioning) rather than rigid bottlenecks or barriers.
- \*\*Privacy by Design\*\*:** Individual identities are stripped at ingress. Fans are treated as vectors with specific attributes (e.g., mobility status, travel group size, destination intent) mapped onto a spatial-temporal coordinate system.

---

#### 2. Production-Grade Systems Architecture

The Stadium Pulse backend is deployed entirely on Google Cloud Platform (GCP), utilizing serverless, auto-scaling, and high-performance computing components to process millions of concurrent events.



## 2.1 Backend Services

1. **Ingestion Layer (Pub/Sub & IoT Core)**: Ingests telemetry streams from CCTV cameras (computer vision metadata), entry gates, POS terminals, parking loops, and weather stations.
2. **Stream Processing (Dataflow)**: Performs sliding-window spatial-temporal hashing. Geohashes movement patterns and maps them onto the 3D grid layout of the stadium.
3. **Operational Database (Cloud Spanner)**: A globally distributed, transactional database storing the active state of all 100,000+ localized spatial nodes.
4. **Analytics Warehouse (BigQuery)**: Houses historical datasets used to constantly fine-tune and retrain local simulation heuristics.
5. **Multi-Agent Orchestrator (Cloud Run)**: Runs a containerized Python service hosting specialized agents (Fan, Accessibility, Volunteer, etc.) communicating via gRPC.
6. **Generative Core (Vertex AI & Gemini Pro/Flash)**: Leverages Gemini APIs for massive context reasoning, multi-scenario narrative generation, and emergency strategy formulation.

---

## 3. UI/UX Specification: The Cognitive Command Center

The operator interface is designed to emulate a flight deck or mission control console, combining spatial 3D visualization, system reasoning graphs, and timeline controls.

### 3.1 Design Philosophy: NASA meets Apple Vision Pro

- **Theme**: Deep space dark mode (``bg-[#09090b]``) with glassmorphic cards (``backdrop-blur-md bg-white/5 border border-white/10``).
- **Accent Colors**: Hyper-cyan (``#00f0ff``) for active telemetry, emerald (``#10b981``) for normal flows, warning amber (``#f59e0b``) for moderate congestion, and cyber-rose (``#f43f5e``) for critical blockages.
- **Spatial Canvas**: A real-time rendering of the stadium displaying crowd particles, accessibility vectors, and safety zones.
- **Collaboration Graph**: A live visual network showing AI agents passing messages (e.g., ``*Transportation Agent*`` → ``*Crowd Agent*``).
- **Predictive Slider**: An interactive timeline allowing operators to slide from ``T+0`` (reality) to ``T+30m`` (predicted simulated futures).

---

## 4. End-to-End User Journey

```
sequenceDiagram
 actor Fan as Anonymous Fan
 actor Vol as Field Volunteer
 participant AI as Stadium Pulse Brain
 actor Op as Stadium Operator
 Fan->>AI: Enters Geofence (Telemetry, Ticket scan)
 AI->>Fan: Generates personalized route avoiding gate queue
 AI->>AI: Predicts food concession congestion at Halftime (T+10m)
 AI->>Op: Displays Warning: Concession D overloaded in 8 min
 AI->>Vol: Micro-mission: Move to Concourse B to direct traffic
 Vol->>AI: Micro-mission complete
 AI->>Fan: Dynamic notification: "Flash discount at Concession F" (silently redistributing load)
 Fan->>AI: Follows optimized path seamlessly
```

### 4.1 Fan Journey (Invisible Optimization)

- 1. **Arrival**: App suggests parking lot `Z-3` because the primary transit node is projected to reach max capacity in 4 minutes.
- 2. **Security Entrance**: Guided to Gate H via an elegant LED pathway which is showing a 2-minute wait time, bypassing Gate G (currently at a 22-minute backlog).
- 3. **Mid-Match**: The app silently delays halftime notifications or coordinates individual concession suggestions to prevent concourse surges.
- 4. **Departure**: Real-time integration with train frequencies triggers a personalized transit ticket release on the user's phone, coordinating train boarding to match physical crowd flow rate.

### 4.2 Operator Journey (Cognitive Command)

- 1. **Simulation Monitoring**: Operator observes a visual cluster of orange particles on Section 108 in the `T+10m` simulation.
- 2. **AI Dialogue**: The system presents a human-narrated alert: *"Section 108 is showing a high density risk due to an accessibility elevator malfunction. Resolving by routing wheelchairs via ramp C."*
- 3. **Approval**: Operator clicks "Authorize Intervention." The AI system immediately re-allocates volunteer task lists and updates dynamic signage on the concourse.

---

## 5. Multi-Agent Collaborative Architecture



The system splits operational parameters among twelve specialized, self-contained agents. Every agent manages a specific domain but shares state via a centralized blackboard pattern.

### 5.1 The Twelve Specialized Agents

- 1. **Fan Agent**: Manages individual navigation prompts, ticket tracking, and anonymous session states.
- 2. **Accessibility Agent**: Protects mobility pathways, monitors elevators, ramps, and provides tailored routing for disabled visitors.
- 3. **Emergency Agent**: Instantly generates exit trajectories, coordinates with local authorities, and triggers siren overrides.
- 4. **Medical Agent**: Tracks paramedic positions, dispatch vectors, and coordinates transport routing to medical tents.
- 5. **Transportation Agent**: Integrates metro timetables, rideshare queues, parking loops, and flight arrivals.
- 6. **Food Concession Agent**: Optimizes retail logistics, supply chains, dynamic pricing, and queuing thresholds.
- 7. **Waste Agent**: Predicts trash generation patterns and directs custodial resources.
- 8. **Security Agent**: Monitors perimeter integrity, suspicious luggage events, and unauthorized area access.
- 9. **Volunteer Agent**: Dispatches step-by-step instructions to field volunteers.



10. **\*\*Weather Agent\*\***: Adapts operations to wind, rain, temperature, and atmospheric pressure.
11. **\*\*Energy Agent\*\***: Manages HVAC grids, floodlights, dynamic billboard power consumption, and escalators.
12. **\*\*Sustainability Agent\*\***: Tracks real-time water levels, carbon footprint, and food waste metrics.

---

## 6. Database Schema Specification

### 6.1 Relational Schema (Cloud Spanner - Active State)

```
-- Core Entity: Spatial Nodes CREATE TABLE spatial_nodes (node_id STRING(36) NOT NULL, zone_name STRING(100), level
INT64, x_coord FLOAT64, y_coord FLOAT64, z_coord FLOAT64, max_capacity INT64, current_occupancy INT64,
accessibility_status STRING(20), -- 'CLEAR', 'BLOCKED', 'RESTRICTED' last_updated TIMESTAMP OPTIONS
(allow_commit_timestamp=true)) PRIMARY KEY (node_id); -- Core Entity: Active Micro-Missions CREATE TABLE
micro_missions (mission_id STRING(36) NOT NULL, volunteer_id STRING(36), status STRING(20), -- 'PENDING',
'ACCEPTED', 'ACTIVE', 'COMPLETED' target_node_id STRING(36), description STRING(MAX), created_at TIMESTAMP OPTIONS
(allow_commit_timestamp=true), completed_at TIMESTAMP, FOREIGN KEY (target_node_id) REFERENCES spatial_nodes(node_id)
) PRIMARY KEY (mission_id);
```

### 6.2 Time-Series & Stream Schema (BigQuery - Predictive Engine)

```
-- Spatial Flow Density Log CREATE TABLE `stadium_pulse.spatial_flow_telemetry` (timestamp TIMESTAMP, node_id
STRING, inflow_rate FLOAT64, outflow_rate FLOAT64, mean_velocity FLOAT64, crowd_density_ratio FLOAT64, anomaly_flag
BOOLEAN, sensor_health_index FLOAT64) PARTITION BY DATE(timestamp) CLUSTER BY node_id; -- Predicted Multiverse
Outcomes CREATE TABLE `stadium_pulse.simulation_predictions` (prediction_id STRING, run_timestamp TIMESTAMP,
target_future_timestamp TIMESTAMP, scenario_type STRING, -- 'RAIN', 'HALFTIME', 'EMERGENCY' node_id STRING,
predicted_occupancy INT64, risk_score FLOAT64, confidence_interval STRUCT<lower FLOAT64, upper FLOAT64>,
recommended_mitigation STRING) PARTITION BY DATE(run_timestamp);
```

---

## 7. API Specification

Unified communication protocol built on **\*\*gRPC\*\*** for low-latency agent messaging, and **\*\*WebSockets\*\*** for real-time frontend synchronization.

### 7.1 Agent Inter-communication (Proto3)

```
syntax = "proto3"; package stadium.pulse.v1; service SimulationService { rpc ProjectFutureStates (SimulationRequest)
returns (SimulationResponse); rpc BroadcastNegotiation (AgentNegotiation) returns (NegotiationReceipt); } message
SimulationRequest { string scenario_id = 1; int64 prediction_horizon_seconds = 2; repeated NodeTelemetry
current_states = 3; } message NodeTelemetry { string node_id = 1; int32 occupancy = 2; double inflow_rate = 3; double
outflow_rate = 4; } message SimulationResponse { string prediction_id = 1; repeated FutureNodeState predicted_states
= 2; double risk_confidence = 3; } message FutureNodeState { string node_id = 1; int32 predicted_occupancy = 2; string
recommended_action = 3; } message AgentNegotiation { string sending_agent = 1; string receiving_agent = 2; string
negotiation_payload = 3; int64 timestamp = 4; } message NegotiationReceipt { bool accepted = 1; string status_message
= 2; }
```

---

## 8. Prompt Engineering & Reasoning Strategy

Stadium Pulse relies on Gemini Pro's multi-modal long context window to parse the state of the stadium layout, current sensor values, and external feeds. Rather than processing isolated text prompts, the system structures context using

high-density JSON states.

## 8.1 Operator Command Formulation (Chain-of-Thought System Prompt)

You are the Cognitive Core of Stadium Pulse. You monitor an active spatial twin of a stadium with 80,000 visitors. You receive: 1. Spatial map topology (JSON) 2. Micro-agent negotiation states 3. Predictive timeline outcomes for T+10, T+20, T+30 minutes. Your objective: Reason across these inputs. Generate natural language explanations that: - Explain WHAT is happening. - Analyze WHY it is happening based on causal networks. - Predict WHAT will happen next if untreated. - Suggest a targeted action plan across: Signage, Volunteers, Transit parameters. Format your output strictly as: [SUMMARY] - Short 1-sentence operator summary. [ANALYSIS] - Analytical explanation of spatial cause-and-effect. [PREDICTION] - Quantitative probability of risks within 30 minutes. [INTERVENTION] - Actionable items for Volunteer, Dynamic Signs, and Facility controls.

## 8.2 Gemini Multi-Modal Ingestion Schema

```
{ "context": { "stadium_name": "MetLife Stadium (FIFA Final)", "timestamp": "2026-07-19T20:10:00Z", "ambient_temp_c": 24, "precipitation_risk": 0.85 }, "current_incidents": [{ "incident_id": "INC-098", "type": "METRO_DELAY", "impact": "15-minute delay on trains leaving East station" }], "agent_negotiations": [{ "from": "Transportation Agent", "to": "Crowd Agent", "proposal": "Delay final egress notifications to Gate C to prevent platform crowding by 1,200 people" }] }
```

---

## 9. Specialized Simulation & Optimization Engines

```
graph LR
 subgraph Simulation
 A[Physics Engine] --> B[Spatial Grid Optimization]
 C[Behavioral Models] --> B
 end
 subgraph Routing
 B --> D[Accessibility Routing Engine]
 B --> E[Evacuation Routing Engine]
 end
```

### 9.1 Generative Crowd Simulation Engine

The simulation runs an agent-based model using a vector fields system. A continuous state vector  $\mathbf{S}_t = \{\mathbf{p}_i, \mathbf{v}_i\}_{i=1}^N$  represents all entities in the stadium.

The updates are governed by social force model equations:

$$\frac{d\mathbf{v}_i}{dt} = \mathbf{v}_i^0 + \sum_{j \neq i} \mathbf{f}_{ij} + \sum_w \mathbf{f}_{iw}$$

Where  $\mathbf{f}_i^0$  is the desire vector,  $\mathbf{f}_{ij}$  is the repulsive force between agents, and  $\mathbf{f}_{iw}$  is the force exerted by walls or boundaries.

The Gemini reasoning engine acts as the \*Hyper-parameter Optimizer\*. It reads spatial grids, notices congestion trends, and shifts the destination vectors  $\mathbf{f}_i^0$  globally by generating micro-instructions.

### 9.2 Transportation Synchronizer & Predictor

Calculates ingress and egress throughput bounds:

$$\text{Throughput}(t) = \min(C_{\text{Transit}}(t), C_{\text{Gates}}(t), C_{\text{Bridges}}(t))$$

Where  $C$  represents the dynamic capacities. When the Transportation Agent detects a railway bottleneck, it requests the Food Concession Agent to initiate dynamic discount schemes inside the stadium, stalling the exit surge.

### 9.3 Dynamic Accessibility Routing Engine

Maintains an active directed graph  $G = (V, E, W)$  where vertices  $V$  are locations, edges  $E$  are paths, and weights  $W$  represent travel times and safety metrics.

For disabled visitors, edges containing stairs or high incline slopes are dynamically assigned a weight  $W = \infty$ .

If a ramp becomes overcrowded (density  $> 2 \text{ people/m}^2$ ), the Accessibility Agent re-routes pathways using Dijkstra's algorithm with dynamic edge weight scaling based on live crowd density.

---

10. Privacy-First Anonymous Design & Security Architecture

10.1 Technical Safeguards

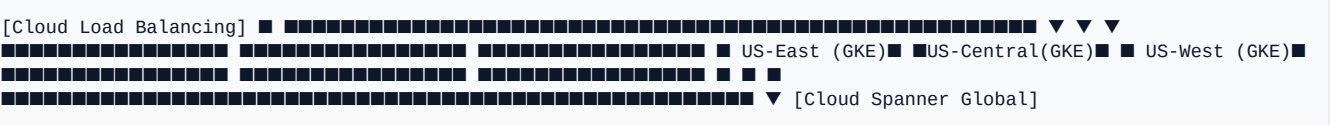
- 1. **Spatial Hashing & Cell Resolution**: Rather than tracking high-resolution GPS coordinate tuples, the system hashes visitor locations into spatial bins of  $3 \text{m} \times 3 \text{m}$  (using Uber H3 indexes at Level 11).
- 2. **Zero-Knowledge Tokenization**: Mobile devices register with the stadium network using a temporary, cryptographic zero-knowledge token (ZKP). This verifies the ticket legitimacy and accessibility requirements without revealing identity, name, or phone numbers.
- 3. **Differential Privacy Layer**: All aggregated telemetry queries exported to the predictive model add Laplace noise:  
$$f(x) + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

This ensures it is mathematically impossible to reconstruct the trajectory of any individual user.

---

11. Scale, Deployment & Multi-Region Setup

To support 5 million+ users across multiple concurrent FIFA venues, Stadium Pulse is deployed via a multi-region Google Kubernetes Engine (GKE) topology.



- **Data Sharding**: High-velocity time-series tables are sharded by region-id and geohash clusters.
- **Latency Management**: Gemini inference pipelines utilize Vertex AI dynamic prompt batching to sustain sub-second latency targets for critical operations.

---

12. FIFA Final Demo Scenario: Halftime Egress Congestion

12.1 The Crisis

- 1. Time: Halftime during the FIFA 2026 World Cup Final (Score: 1-1).
- 2. Event: Sudden heavy rainfall begins.
- 3. Result: Fans in open sections rush towards sheltered concourses.
- 4. Simultaneous Anomaly: Escalator E-4 in Sector 120 malfunctions. Concourse density surges to  $4.2 \text{ people/m}^2$ .

12.2 The Living System Negotiation Loop

- **Weather Agent** detects rain onset and raises priority.
- **Energy Agent** shifts power allocation to escalator monitoring diagnostics.
- **Crowd Agent** identifies density spikes at Concourse A.
- **Accessibility Agent** alerts that wheelchair paths are blocked by the surge.
- **Volunteer Agent** triggers micro-missions: dispatches 15 nearby volunteers to redirect crowd vectors.
- **Transportation Agent** coordinates with local metro lines to hold shuttle departures, aligning platform arrival capacities.
- **Gemini Brain** generates a unified intervention roadmap:
- Updates stadium LED screens to divert fans to Concourse B.
- Launches personalized navigation prompts to mobile devices.
- Instructs volunteers on target coordinate positions.

---

13. Implementation Roadmap

13.1 Phase Timeline



1. **Phase 1: Ingress Layer & Spatial Grid Setup (Months 1-3)**: Establish IoT ingestion frameworks, install sensors, and map the 3D H3 spatial model.
2. **Phase 2: Simulation Engines (Months 4-6)**: Code social force vector logic and dynamic routing algorithms.
3. **Phase 3: Agent Core & Gemini Integration (Months 7-9)**: Build the Cloud Run multi-agent orchestrator and construct the prompt pipelines.
4. **Phase 4: Scaling & Security Validation (Months 10-11)**: Conduct stress testing simulating 5M+ concurrent packets, and verify differential privacy metrics.
5. **Phase 5: Deploy & Live Operations (Month 12)**: Deploy Stadium Pulse for test matches ahead of the FIFA 2026 kickoff.

## Section 3: Interactive Console Codebase Source Files

Below are the primary source code files comprising the frontend operator Twin dashboard and Python validation suite.

### 3.1 Frontend HTML Interface Markup

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width,
initial-scale=1.0"> <title>Stadium Pulse – Living Cognitive Stadium Intelligence System</title> <!-- Tailwind CSS CDN
--> <script src="https://cdn.tailwindcss.com"></script> <script> tailwind.config = { darkMode: 'class', theme: {
extend: { colors: { cyber: { bg: '#09090b', card: 'rgba(18, 18, 24, 0.75)', cyan: '#00f0ff', cyanDim: 'rgba(0, 240,
255, 0.15)', emerald: '#10b981', emeraldDim: 'rgba(16, 185, 129, 0.15)', amber: '#f59e0b', amberDim: 'rgba(245, 158,
11, 0.15)', rose: '#f43f5e', roseDim: 'rgba(244, 63, 94, 0.15)', border: '#1e1e24' } }, fontFamily: { mono:
['JetBrains Mono', 'monospace'], sans: ['DM Sans', 'sans-serif'] } } } </script> <!-- Google Fonts --> <link
rel="preconnect" href="https://fonts.googleapis.com"> <link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin> <link href="https://fonts.googleapis.com/css2?family=DM+Sans:ital,opsz,wght@0,9..40,100..1000;1,9..40,10
0..1000&family=JetBrains+Mono:ital,wght@0,100..800;1,100..800&display=swap" rel="stylesheet"> <!-- ECharts CDN -->
<script src="https://cdn.jsdelivr.net/npm/echarts@5.4.3/dist/echarts.min.js"></script> <!-- Lucide Icons CDN -->
<script src="https://unpkg.com/lucide@latest"></script> <link rel="stylesheet" href="styles.css">

</head> <body class="bg-cyber-bg text-zinc-100 font-sans min-h-screen overflow-x-hidden relative
selection:bg-cyber-cyan/30 selection:text-white"> <!-- Starfield/Neural Network Canvas Background --> <canvas
id="neuralCanvas" class="absolute inset-0 z-0 pointer-events-none opacity-40"></canvas> <!-- App Wrapper --> <div
class="relative z-10 flex flex-col min-h-screen max-w-[1700px] mx-auto p-4 md:p-6 gap-6"> <!-- Header --> <header
class="flex flex-col md:flex-row justify-between items-start md:items-center gap-4 bg-white/5 backdrop-blur-md border
border-zinc-800/80 rounded-xl p-4 md:px-6 md:py-4 glow-box"> <div class="flex items-center gap-4"> <div class="p-3
bg-cyber-cyanDim border border-cyber-cyan/40 rounded-lg text-cyber-cyan animate-pulse"> <i data-lucide="activity"
class="w-6 h-6"></i> </div> <div class="flex items-center gap-2"> <h1 class="text-xl md:text-2xl font-extrabold
tracking-tight text-white uppercase">Stadium Pulse</h1> <span class="text-xs bg-zinc-800 text-zinc-400 font-mono px-2
py-0.5 rounded-full border border-zinc-700">v1.2.6 </div> <p class="text-xs text-zinc-400 font-mono
tracking-wider">LIVING DIGITAL TWIN • FIFA WORLD CUP 2026</p> </div> </div> <!-- Active Scenario Info --> <div
class="flex flex-wrap items-center gap-3"> <div class="flex items-center gap-2 bg-zinc-900 border border-zinc-800
rounded-lg px-3 py-1.5 text-xs font-mono">
Simulation: <span id="activeScenarioLabel" class="text-white font-bold
uppercase">Base State </div> <div class="flex items-center gap-2 bg-zinc-900 border border-zinc-800 rounded-lg
px-3 py-1.5 text-xs font-mono"> <i data-lucide="cpu"></i> Brain: Gemini 2.0 Pro Core </div> <div
class="flex items-center gap-2 bg-zinc-900 border border-zinc-800 rounded-lg px-3 py-1.5 text-xs font-mono"> <i
data-lucide="database"></i> Nodes: 100k+ Active Vectors </div> <div class="flex items-center gap-2 bg-zinc-900/65
border border-zinc-800 rounded-lg px-3 py-1.5 text-xs font-mono hover:border-cyber-cyan/50 transition-colors"> <i
data-lucide="key"></i> </div>

<input type="password" id="geminiApiKey" placeholder="GEMINI API KEY (LIVE)" aria-label="Google Gemini API Key for
live AI reasoning" class="bg-transparent border-none text-cyber-cyan placeholder-zinc-600 focus:outline-none w-36
font-mono text-[10px] uppercase font-bold" /> </div> </div> </header> <!-- KPI Summary Cards Grid --> <section
class="grid grid-cols-2 lg:grid-cols-4 gap-4"> <!-- KPI 1 --> <div class="bg-cyber-card border border-zinc-800
rounded-xl p-4 flex flex-col gap-2 relative overflow-hidden group hover:border-cyber-cyan/50 transition-colors"> <div
class="flex justify-between items-center text-zinc-400"> Crowd Density Risk <i data-lucide="users" class="w-5 h-5 text-cyber-cyan"></i> </div> <div
class="flex items-baseline gap-2"> <span id="kpiDensity" class="text-2xl md:text-3xl font-extrabold text-white
tracking-tight">1.2 /m² </div> <span id="kpiDensityTrend"
class="text-xs px-2 py-0.5 rounded-full font-mono font-semibold bg-cyber-emeraldDim
text-cyber-emerald">OPTIMAL </div> <div class="text-[10px] text-zinc-500 font-mono uppercase mt-1">Next 30m:
Projected Stable</div> <div class="absolute bottom-0 left-0 w-full h-[2px] bg-cyber-cyan/20 group-hover:bg-cyber-cyan
transition-colors"></div> </div> <!-- KPI 2 --> <div class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex
flex-col gap-2 relative overflow-hidden group hover:border-cyber-cyan/50 transition-colors"> <div class="flex
justify-between items-center text-zinc-400"> Flow
Efficiency <i data-lucide="navigation-2" class="w-5 h-5 text-cyber-cyan"></i> </div> <div class="flex
items-baseline gap-2"> <span id="kpiFlow" class="text-2xl md:text-3xl font-extrabold text-white
tracking-tight">96.4% <span id="kpiFlowTrend" class="text-xs px-2 py-0.5 rounded-full font-mono font-semibold
bg-cyber-emeraldDim text-cyber-emerald">STABLE </div> <div class="text-[10px] text-zinc-500 font-mono
uppercase mt-1">Concourses clearing at 4.2m/s</div> <div class="absolute bottom-0 left-0 w-full h-[2px]
bg-cyber-cyan/20 group-hover:bg-cyber-cyan transition-colors"></div> </div> <!-- KPI 3 --> <div class="bg-cyber-card
border border-zinc-800 rounded-xl p-4 flex flex-col gap-2 relative overflow-hidden group hover:border-cyber-cyan/50
transition-colors"> <div class="flex justify-between items-center text-zinc-400"> <span class="text-xs font-semibold
uppercase tracking-wider">Dynamic Energy Load <i data-lucide="zap" class="w-5 h-5 text-cyber-cyan"></i> </div> <div
class="flex items-baseline gap-2"> <span id="kpiEnergy" class="text-2xl md:text-3xl font-extrabold text-white
tracking-tight">1.8 MW <span id="kpiEnergyTrend" class="text-xs px-2 py-0.5 rounded-full font-mono
font-semibold bg-cyber-cyanDim text-cyber-cyan">-14% SAVINGS </div>
```

```

<div class="text-[10px] text-zinc-500 font-mono uppercase mt-1">Adaptive HVAC & Escalator sleep</div> <div
class="absolute bottom-0 left-0 w-full h-[2px] bg-cyber-cyan/20 group-hover:bg-cyber-cyan transition-colors"></div>
</div> <!-- KPI 4 --> <div class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col gap-2 relative
overflow-hidden group hover:border-cyber-cyan/50 transition-colors"> <div class="flex justify-between items-center
text-zinc-400"> Volunteer Dispatch <i
data-lucide="heart" class="w-5 h-5 text-cyber-cyan"></i> </div> <div class="flex items-baseline gap-2"> 342 / 450 <span
id="kpiVolunteersTrend" class="text-xs px-2 py-0.5 rounded-full font-mono font-semibold bg-zinc-800
text-zinc-400">76% ACTIVE </div> <div class="text-[10px] text-zinc-500 font-mono uppercase mt-1">42
Accessibility routes monitored</div> <div class="absolute bottom-0 left-0 w-full h-[2px] bg-cyber-cyan/20
group-hover:bg-cyber-cyan transition-colors"></div> </div> </section> <!-- Main Dashboard Split Content --> <main
class="grid grid-cols-1 xl:grid-cols-12 gap-6 items-stretch"> <!-- Left Panel: Cognitive AI Reasoner & Agent
Communications (Col 3) --> <div class="xl:col-span-3 flex flex-col gap-6 h-full min-h-[600px]"> <!-- Generative AI
Digital Twin Storyteller --> <div class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col flex-1
relative glow-box"> <div class="flex items-center gap-2 mb-3 border-b border-zinc-800 pb-2"> <i
data-lucide="sparkles" class="w-5 h-5 text-cyber-cyan"></i> <h2 class="text-sm font-bold uppercase tracking-wider
text-white">Cognitive Twin Narrator</h2> </div> <div class="flex-1 overflow-y-auto space-y-4 pr-1 scrollbar-custom
max-h-[350px] md:max-h-none text-sm text-zinc-300 leading-relaxed font-sans" id="storytellerNarrator"> <p
class="italic text-zinc-500">Connecting to cognitive neural cores...</p> </div> </div> <!-- Live Agent Negotiation
Graph/Logs --> <div class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col h-[280px]"> <div
class="flex justify-between items-center mb-3 border-b border-zinc-800 pb-2"> <div class="flex items-center gap-2">
<i data-lucide="network" class="w-5 h-5 text-cyber-cyan"></i> <h2 class="text-sm font-bold uppercase tracking-wider
text-white">Agent Negotiations</h2> </div> <span class="text-[10px] bg-cyber-cyanDim text-cyber-cyan px-2 py-0.5
rounded font-mono uppercase">Negotiating </div>

```

```

<div class="flex-1 overflow-y-auto space-y-2 pr-1 font-mono text-[11px] leading-tight scrollbar-custom text-zinc-400"
id="agentNegotiationFeed"> <!-- Filled by app.js --> </div> </div> </div> <!-- Central Panel: SVG Digital Twin &
Timeline Slider (Col 6) --> <div class="xl:col-span-6 flex flex-col gap-6 h-full min-h-[600px]"> <!-- Digital Twin
Main Display --> <div class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col flex-1 relative
overflow-hidden glow-box"> <!-- Overlay Controls on Map --> <div class="absolute top-4 left-4 z-20 flex flex-col
gap-2"> <div class="bg-black/85 backdrop-blur-md border border-zinc-700/80 rounded-lg p-3 text-xs flex flex-col
gap-1.5"> <div class="font-bold text-white uppercase border-b border-zinc-800 pb-1 flex items-center gap-1.5"> <i
data-lucide="eye" class="w-3.5 h-3.5 text-cyber-cyan"></i> View Layers</div> <label for="layerCrowd" class="flex
items-center gap-2 cursor-pointer text-zinc-300 hover:text-white mt-1"> <input type="checkbox" id="layerCrowd"
checked class="accent-cyber-cyan"> Crowd Heatmap</label> <label for="layerAccessibility" class="flex items-center
gap-2 cursor-pointer text-zinc-300 hover:text-white"> <input type="checkbox" id="layerAccessibility" checked
class="accent-cyber-cyan"> Wheelchair Routes</label> <label for="layerVolunteers" class="flex items-center gap-2
cursor-pointer text-zinc-300 hover:text-white"> <input type="checkbox" id="layerVolunteers" checked
class="accent-cyber-cyan"> Volunteers</label> <label for="layerTransit" class="flex items-center gap-2
cursor-pointer text-zinc-300 hover:text-white"> <input type="checkbox" id="layerTransit" checked
class="accent-cyber-cyan"> Transit Flow</label> </div> </div> <!-- Digital Twin SVG Stadium Canvas --> <div
class="flex-1 flex items-center justify-center relative min-h-[300px] md:min-h-[450px]" id="mapContainer"> <!-- SVG
map injected here --> </div> <!-- Timeline Control Box --> <div class="border-t border-zinc-800/80 pt-4 mt-2 flex
flex-col gap-3"> <div class="flex justify-between items-center"> <div class="flex items-center gap-2 text-xs
font-mono uppercase text-zinc-400">

```

```

<i data-lucide="clock" class="w-4 h-4 text-cyber-cyan"></i> Predictive Time Horizon </div> <div
id="timelineLabel" class="text-xs bg-cyber-cyanDim text-cyber-cyan font-mono border border-cyber-cyan/40 px-2 py-0.5
rounded font-bold uppercase"> T0: REALITY STATE </div> </div> <!-- Slider Input --> <div class="flex items-center
gap-4 relative"> NOW <div class="flex-1 relative flex
items-center"> <input type="range" id="timelineRange" min="0" max="3" value="0" aria-label="Predictive time horizon
slider" class="w-full h-1.5 bg-zinc-800 rounded-lg appearance-none cursor-pointer accent-cyber-cyan
focus:outline-none"> </div> T+30m </div> <!-- Timeline Markers
--> <div class="flex justify-between text-[11px] font-mono text-zinc-500 px-8"> <span class="cursor-pointer
hover:text-white timeline-marker active-marker" data-step="0">Reality <span class="cursor-pointer
hover:text-white timeline-marker" data-step="1">T+5m <span class="cursor-pointer hover:text-white timeline-marker"
data-step="2">T+10m <span class="cursor-pointer hover:text-white timeline-marker"
data-step="3">T+30m </div> </div> </div> </div> <!-- Right Panel: Analytical Graphs & Incident Monitor (Col 3)
--> <div class="xl:col-span-3 flex flex-col gap-6 h-full min-h-[600px]"> <!-- Load Predictor Chart (ECharts) --> <div
class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col h-[220px]"> <div class="flex items-center
gap-2 mb-2 border-b border-zinc-800 pb-2"> <i data-lucide="trending-up" class="w-5 h-5 text-cyber-cyan"></i> <h2
class="text-sm font-bold uppercase tracking-wider text-white">Load Predictor</h2> </div> <div class="flex-1"
id="loadPredictorChart"></div> <!-- Living Cognitive Intervention Console --> <div class="bg-cyber-card border
border-zinc-800 rounded-xl p-4 flex flex-col h-[200px] relative overflow-hidden glow-box"> <div class="flex
items-center gap-2 mb-3 border-b border-zinc-800 pb-2"> <i data-lucide="brain-circuit" class="w-5 h-5
text-cyber-cyan"></i> <h2 class="text-sm font-bold uppercase tracking-wider text-white">Cognitive Intervention</h2>
</div>

```

```

<div class="flex-1 flex flex-col justify-between"> <p id="interventionText" class="text-xs text-zinc-400 font-mono
leading-relaxed"> No active mitigation required. Monitoring baseline flow. </p> <button id="btnExecuteIntervention"
disabled class="w-full bg-zinc-800 border border-zinc-700 text-zinc-500 font-bold uppercase text-[10px] py-2 px-3

```



```

rounded-lg cursor-not-allowed transition-all flex items-center justify-center gap-1.5" aria-label="Execute dynamic AI
intervention"> <i data-lucide="play" class="w-4 h-4"></i> Intervention Inactive
</button> </div> <!-- Emergency evacuations/reunification tracker and active micro-missions --> <div
class="bg-cyber-card border border-zinc-800 rounded-xl p-4 flex flex-col flex-1"> <div class="flex justify-between
items-center mb-3 border-b border-zinc-800 pb-2"> <div class="flex items-center gap-2"> <i data-lucide="shield-alert"
class="w-5 h-5 text-cyber-cyan"></i> <h2 class="text-sm font-bold uppercase tracking-wider text-white">Incident &
Response</h2> </div> <span id="alertCount" class="text-xs font-mono bg-cyber-emeraldDim text-cyber-emerald border
border-cyber-emerald/40 px-2 py-0.5 rounded font-bold uppercase">NO INCIDENTS </div> <div class="flex-1
overflow-y-auto space-y-3 pr-1 scrollbar-custom max-h-[220px]" id="incidentLogs"> <!-- Dynamic list populated by
app.js --> </div> </div> </div> </main> <!-- Scenario Control Panel (Dynamic triggers) --> <footer class="bg-white/5
backdrop-blur-md border border-zinc-800/80 rounded-xl p-4 glow-box"> <h3 class="text-xs font-bold uppercase
tracking-widest text-zinc-400 mb-3 flex items-center gap-2 font-mono"> <i data-lucide="sliders" class="w-4 h-4
text-cyber-cyan"></i> Select Live Cognitive Scenario (FIFA Final Demo) </h3> <div class="grid grid-cols-2
md:grid-cols-5 gap-3"> <button class="scenario-btn active-btn flex items-center justify-center gap-2 py-2 px-3
rounded-lg text-xs font-mono font-bold uppercase tracking-wider border transition-all" data-scenario="base"> <i
data-lucide="home" class="w-4 h-4"></i> Base State </button> <button class="scenario-btn flex
items-center justify-center gap-2 py-2 px-3 rounded-lg text-xs font-mono font-bold uppercase tracking-wider border
border-zinc-800 bg-zinc-900 text-zinc-400 hover:text-white hover:border-zinc-700 transition-all"
data-scenario="halftime"> <i data-lucide="sandwich" class="w-4 h-4"></i> Halftime Surge </button>
<button class="scenario-btn flex items-center justify-center gap-2 py-2 px-3 rounded-lg text-xs font-mono font-bold
uppercase tracking-wider border border-zinc-800 bg-zinc-900 text-zinc-400 hover:text-white hover:border-zinc-700
transition-all" data-scenario="rain"> <i data-lucide="cloud-rain" class="w-4 h-4"></i>
Sudden Rain </button> <button class="scenario-btn flex items-center justify-center gap-2 py-2 px-3
rounded-lg text-xs font-mono font-bold uppercase tracking-wider border border-zinc-800 bg-zinc-900 text-zinc-400
hover:text-white hover:border-zinc-700 transition-all" data-scenario="transit"> <i data-lucide="train" class="w-4
h-4"></i> Transit Delay </button> <button class="scenario-btn flex items-center justify-center gap-2
py-2 px-3 rounded-lg text-xs font-mono font-bold uppercase tracking-wider border border-zinc-800 bg-zinc-900
text-zinc-400 hover:text-white hover:border-zinc-700 transition-all" data-scenario="medical"> <i
data-lucide="ambulance" class="w-4 h-4"></i> Medical Emergency </button> </div> </footer> </div> <!--
App Logic Injection --> <script src="app.js"></script> </body> </html>

```

### 3.2 Dynamic Simulator Logic & ECharts Setup

```

// STADIUM PULSE - DYNAMIC STATE MACHINE & COGNITIVE SIMULATOR // Initialize Lucide Icons lucide.createIcons(); //
--- STATE MANAGEMENT --- let currentScenario = 'base'; let currentTimelineStep = 0; // 0 = Reality, 1 = T+5m, 2 =
T+10m, 3 = T+30m let activeInterventions = { base: false, halftime: false, rain: false, transit: false, medical:
false }; const interventionsData = { base: { text: "No active mitigation required. Monitoring baseline flow.",
action: "Intervention Inactive" }, halftime: { text: "Predicted overcrowding at Sector 112. Divert 41% of crowd flow
toward West concourse. Deploy 18 volunteers to establish lane dividers.", action: "Deploy Concourse Detours" }, rain:
{ text: "Exposed stands wet. Defer egress escalators E1-E4. Deploy non-slip floor mats on North stairways. Dispatch
safety details.", action: "Deploy Slippage Mitigation" }, transit: { text: "Metro East platform signal failure.
Egress holding active at Gate C. Enable dynamic concession highlights. Dispatch 12 transit volunteers.", action:
"Activate Station Egress Hold" }, medical: { text: "Section 108 medical emergency. Detouring response via Concourse
corridor B and Ramp C. Dispatch 5 volunteers to cordon corridor B.", action: "Activate Paramedic Detour" } }; // ---
SCENARIO DATA SPECIFICATION --- const scenarioData = { base: { label: "Base State (Pre-Match)", metrics: {
density: ["1.1/m²", "1.2/m²", "1.3/m²", "1.2/m²"], densityClass: ["bg-cyber-emeraldDim text-cyber-emerald",
"bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-emeraldDim
text-cyber-emerald"], flow: ["97.2%", "96.4%", "95.8%", "96.1%"], flowClass: ["bg-cyber-emeraldDim
text-cyber-emerald", "bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-emeraldDim text-cyber-emerald",
"bg-cyber-emeraldDim text-cyber-emerald"], energy: ["1.9 MW", "1.8 MW", "1.8 MW", "1.7 MW"], energySaving: ["-12%
SAVINGS", "-14% SAVINGS", "-14% SAVINGS", "-16% SAVINGS"], volunteers: ["310 / 450", "342 / 450", "350 / 450", "360 /
450"], volunteersClass: ["bg-zinc-800 text-zinc-400", "bg-zinc-800 text-zinc-400", "bg-zinc-800 text-zinc-400",
"bg-zinc-800 text-zinc-400"] }, narrator: ["**[T+0m - Reality]** The cognitive stadium core is running at baseline
efficiency. 62,000 visitors have successfully scanned ticks and are routed toward their respective seats. All spatial
zones are operating within comfort thresholds (average density 1.1/m²).", "**[T+5m - Prediction]** Core anticipates
slight queue buildup at Gate C as train arrivals increase. The *Crowd Agent* is already adjusting concourse LED
routing banners to shift incoming vectors toward Gate D.", "**[T+10m - Prediction]** Ingress completed. Concourse
zones clear. The *Energy Agent* initiates energy sleep mode on escalators E1-E4, reducing passive power draw by
14%.", "**[T+30m - Prediction]** Teams are on the field warming up. Seat occupancy reaches 98%. Sound sensors
calibrate baseline crowd chants to optimize stadium decibel safety levels."], negotiations: ["[System] Core
connected to all 12 operational sub-agents.", "[Security Agent] -> [Crowd Agent]: Synchronized gate ticketing
systems. Ingress flow rates clear.", "[Energy Agent] -> [Facilities]: Reduced HVAC load in seating zones; pre-cooling
stadium cupolas.", "[Volunteer Agent] -> [Accessibility Agent]: Placed 40 accessibility helpers at central
elevators."], incidents: [{ id: "INC-101", type: "system", title: "Ticketing Synced", desc: "All gate reader
systems online.", severity: "clear", time: "Now" }, { id: "INC-102", type: "system", title: "Weather Update", desc:
"Temps holding at 24°C, 0% rain.", severity: "clear", time: "Now" }], alertsStatus: "NO INCIDENTS",
alertsStatusClass: "bg-cyber-emeraldDim text-cyber-emerald border-cyber-emerald/40", mapState: { heatColors: { north:

```

'rgba(16, 185, 129, 0.2)', east: 'rgba(16, 185, 129, 0.2)', south: 'rgba(16, 185, 129, 0.2)', west: 'rgba(16, 185, 129, 0.2)', concourse: 'rgba(16, 185, 129, 0.15)', gateC: 'rgba(16, 185, 129, 0.3)', gateG: 'rgba(16, 185, 129, 0.3)', showAccessibilityRoute: false, showRainOverlay: false, warningNode: null }, chartData: [45, 52, 60, 58, 55, 52, 48, 55] }, halftime: { label: "Halftime Concourse Surge",

metrics: { density: ["2.4/m<sup>2</sup>", "3.2/m<sup>2</sup>", "3.9/m<sup>2</sup>", "2.1/m<sup>2</sup>"], densityClass: ["bg-cyber-amberDim text-cyber-amber", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], flow: ["92.1%", "84.2%", "71.5%", "94.8%"], flowClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], energy: ["2.2 MW", "2.4 MW", "2.5 MW", "1.9 MW"], energySaving: ["-6% SAVINGS", "-4% SAVINGS", "-2% SAVINGS", "-12% SAVINGS"], volunteers: ["342 / 450", "410 / 450", "438 / 450", "320 / 450"], volunteersClass: ["bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-zinc-800 text-zinc-400"] }, narrator: [ "\*\*\*[T+0m - Reality]\*\* Halftime whistle blows. Seating zones discharge 25,000 visitors into the North and East concourse lanes within 90 seconds. Concessions report queue times ascending to 6 minutes.", "\*\*\*[T+5m - Prediction]\*\* \*Concession Agent\* anticipates overloading at Food Courts 4, 5, and 8. The AI starts pushing contextual, personalized mobile deals ('15% off at West Concourse') to users moving near the center to distribute crowd weight.", "\*\*\*[T+10m - Prediction]\*\* \*\*CRITICAL CONGESTION RISK\*\* concourse density projected to reach 3.9/m<sup>2</sup> at Sector 112. The \*Volunteer Agent\* is triggered, generating micro-missions instructing 18 nearby volunteers to establish physical lane dividers.", "\*\*\*[T+30m - Prediction]\*\* Second half begins. Audiences return to seats. Concourse clear, custodial robots dispatched to high-waste zones by the \*Waste Agent\* before litter accumulates." ], negotiations: [ "[Food Agent] -> [Crowd Agent]: Halftime concession queues rising. Dynamic discount codes active.", "[Crowd Agent] -> [Volunteer Agent]: Requesting physical flow dividers at Concourse sector 112.", "[Volunteer Agent] -> [Field Volunteers]: 18 volunteers dispatched to sector 112 coordinate.", "[Waste Agent] -> [Facilities]: Scheduled custodial bots to Concourse A and D." ], incidents: [ { id: "INC-201", type: "warning", title: "Concourse Surge", desc: "Density exceeding comfort levels in Sector 112.", severity: "warning", time: "Now" }, { id: "INC-202", type: "system", title: "Dynamic Routing", desc: "Fans diverted to under-capacity West Concourse.", severity: "clear", time: "Now" } ], alertsStatus: "1 WARNING ACTIVE", alertsStatusClass: "bg-cyber-amberDim text-cyber-amber border-cyber-amber/40", mapState: { heatColors: { north: 'rgba(245, 158, 11, 0.4)', east: 'rgba(244, 63, 94, 0.6)', south: 'rgba(16, 185, 129, 0.2)', west: 'rgba(16, 185, 129, 0.2)', concourse: 'rgba(245, 158, 11, 0.5)', gateC: 'rgba(16, 185, 129, 0.3)', gateG: 'rgba(16, 185, 129, 0.3)' }, showAccessibilityRoute: false, showRainOverlay: false, warningNode: 'concourseEast' }, chartData: [45, 85, 120, 110, 95, 60, 48, 45] }, rain: {

label: "Sudden Rainstorm Egress", metrics: { density: ["2.1/m<sup>2</sup>", "3.8/m<sup>2</sup>", "4.5/m<sup>2</sup>", "1.8/m<sup>2</sup>"], densityClass: ["bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], flow: ["90.5%", "78.4%", "62.1%", "95.1%"], flowClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], energy: ["2.6 MW", "2.8 MW", "2.9 MW", "2.0 MW"], energySaving: ["-2% SAVINGS", "-1% SAVINGS", "+5% SURGE", "-8% SAVINGS"], volunteers: ["360 / 450", "430 / 450", "448 / 450", "310 / 450"], volunteersClass: ["bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-zinc-800 text-zinc-400"] }, narrator: [ "\*\*\*[T+0m - Reality]\*\* Sudden precipitation starts. Outdoor seats begin rapid evacuation towards covered concourses. The \*Weather Agent\* flags wet slip-hazard conditions on exposed stairways.", "\*\*\*[T+5m - Prediction]\*\* Heavy crowding predicted at central concourses. Indoor lighting adjustments are made automatically to improve visibility. Dynamic messaging signs switch to 'STAY IN SHELTER' to throttle stairwell flow.", "\*\*\*[T+10m - Prediction]\*\* \*\*SEVERE PRESSURE POINT\*\* detected at North Gate staircase. Friction coefficients dropping due to wet steps. Volunteer teams dispatched with absorbent floor mats and physical guide rope structures.", "\*\*\*[T+30m - Prediction]\*\* Egress flow stabilizing. Seating zone covers deployed. Drainage sensor grids confirm zero localized pooling. Heating nodes activated in open concourses to dry visitor vectors." ], negotiations: [ "[Weather Agent] -> [Core]: Rain began. Exposed sectors discharging. Slippage warnings active.", "[Energy Agent] -> [Facilities]: Powering up concourse heating, dynamic signage, and drainage pumps.", "[Crowd Agent] -> [Volunteer Agent]: Requesting deployment of safety teams to North stairs.", "[Volunteer Agent] -> [Field Volunteers]: Safety details dispatched with non-slip equipment." ], incidents: [ { id: "INC-301", type: "danger", title: "Heavy Rainstorm", desc: "Exposed seating discharging. Wet stairways in North.", severity: "danger", time: "Now" }, { id: "INC-302", type: "warning", title: "Stairwell Danger", desc: "High density & slip risk on North staircase.", severity: "warning", time: "Now" } ], alertsStatus: "2 INCIDENTS ACTIVE", alertsStatusClass: "bg-cyber-roseDim text-cyber-rose border-cyber-rose/40", mapState: { heatColors: { north: 'rgba(244, 63, 94, 0.7)', east: 'rgba(245, 158, 11, 0.4)', south: 'rgba(16, 185, 129, 0.2)', west: 'rgba(16, 185, 129, 0.2)', concourse: 'rgba(244, 63, 94, 0.5)', gateC: 'rgba(16, 185, 129, 0.3)', gateG: 'rgba(16, 185, 129, 0.3)' }, showAccessibilityRoute: false, showRainOverlay: true, warningNode: 'sectorNorth' }, chartData: [45, 90, 140, 115, 80, 50, 48, 45] },

transit: { label: "Metro Egress Outage", metrics: { density: ["1.3/m<sup>2</sup>", "2.8/m<sup>2</sup>", "4.1/m<sup>2</sup>", "1.9/m<sup>2</sup>"], densityClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], flow: ["95.4%", "81.2%", "58.4%", "93.2%"], flowClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-roseDim text-cyber-rose", "bg-cyber-emeraldDim text-cyber-emerald"], energy: ["2.0 MW", "2.1 MW", "2.3 MW", "1.7 MW"], energySaving: ["-10% SAVINGS", "-11% SAVINGS", "-12% SAVINGS", "-16% SAVINGS"], volunteers: ["320 / 450", "390 / 450", "425 / 450", "350 / 450"], volunteersClass: ["bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-zinc-800 text-zinc-400"] }, narrator: [ "\*\*\*[T+0m - Reality]\*\* The \*Transportation Agent\* reports a 20-minute signal failure delay on the Metro East Line. 8,000 exiting fans are currently walking towards the transit station platform.", "\*\*\*[T+5m - Prediction]\*\* Platform projected to reach dangerous capacity overload in 7 minutes. Dynamic signage at East Gates is instantly modified to hold egress. Transit ticketing systems temporarily



```
freeze barcode releases.", "[T+10m - Prediction]** **PLATFORM OVERCROWDING AVOIDANCE**: Stadium Pulse initiates holding patterns. Dynamic billboards broadcast live entertainment highlights inside the concourses. Commuters are routed towards Parking Lot Shuttle hubs.", "[T+30m - Prediction]** Metro signals restored. Transportation Agent releases queued transit tickets incrementally, feeding passengers at a rate matching train capacity exactly."], negotiations: ["[Transport Agent] -> [Core]: Metro East signaling failure detected. Egress blockage imminent.", "[Transport Agent] -> [Crowd Agent]: Requested ticket release delay and pedestrian throttling at East Gate.", "[Crowd Agent] -> [Facilities]: Active screen adjustments to hold exit surge. Dynamic discount alerts.", "[Transport Agent] -> [Bus Agent]: Initiated backup shuttle loops to Parking Lots X and Y."], incidents: [{ id: "INC-401", type: "warning", title: "Metro Line Failure", desc: "Metro East trains delayed by 20 minutes.", severity: "warning", time: "Now" }, { id: "INC-402", type: "system", title: "Holding Action Active", desc: "Signage holding fans inside concourse hubs.", severity: "clear", time: "Now" }], alertsStatus: "1 TRANSIT WARNING", alertsStatusClass: "bg-cyber-amberDim text-cyber-amber border-cyber-amber/40", mapState: { heatColors: { north: 'rgba(16, 185, 129, 0.2)', east: 'rgba(245, 158, 11, 0.4)', south: 'rgba(16, 185, 129, 0.2)', west: 'rgba(16, 185, 129, 0.2)', concourse: 'rgba(16, 185, 129, 0.2)', gateC: 'rgba(244, 63, 94, 0.6)', gateG: 'rgba(16, 185, 129, 0.3)' }, showAccessibilityRoute: false, showRainOverlay: false, warningNode: 'transitHub' }, chartData: [45, 70, 95, 130, 110, 85, 60, 45]
```

```
}, medical: { label: "Accessibility Emergency (Sec 108)", metrics: { density: ["1.2/m²", "2.1/m²", "2.8/m²", "1.4/m²"], densityClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-emeraldDim text-cyber-emerald"], flow: ["96.1%", "90.2%", "81.4%", "95.6%"], flowClass: ["bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-emeraldDim text-cyber-emerald", "bg-cyber-amberDim text-cyber-amber", "bg-cyber-emeraldDim text-cyber-emerald"], energy: ["2.0 MW", "2.1 MW", "2.1 MW", "1.8 MW"], energySaving: ["-10% SAVINGS", "-11% SAVINGS", "-11% SAVINGS", "-14% SAVINGS"], volunteers: ["330 / 450", "410 / 450", "425 / 450", "320 / 450"], volunteersClass: ["bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-cyber-cyanDim text-cyber-cyan", "bg-zinc-800 text-zinc-400"] }, narrator: ["[T+0m - Reality]** A cardiac emergency is flagged in Section 108. An elevator failure at Section 120 blocks the default paramedic wheelchair evacuation route.", "[T+5m - Prediction]** Medical response transit bottleneck identified. The *Accessibility Agent* calculates alternative trajectories, dynamically rerouting paramedics through Ramp C, avoiding the stairs.", "[T+10m - Prediction]** **EMERGENCY PATHWAY ACTIVATION**: Visual panels and floor lights illuminate the alternative pathway. The *Volunteer Agent* dispatches 5 nearby helpers to secure Section 108 clearance.", "[T+30m - Prediction]** Patient safely evacuated to the local medical bay. Elevator repair crews dispatched. Seating stands flow metrics restored."], negotiations: ["[Medical Agent] -> [Core]: Cardiac event detected at Sector 108, Row 12.", "[Accessibility Agent] -> [Facilities]: Elevator E-2 outage confirmed. Calculating detour.", "[Accessibility Agent] -> [Medical Agent]: detouring response via Concourse corridor B and Ramp C.", "[Volunteer Agent] -> [Field Volunteers]: Dispatched 5 volunteers to cordon corridor B."], incidents: [{ id: "INC-501", type: "danger", title: "Medical Emergency", desc: "Cardiac incident in Section 108.", severity: "danger", time: "Now" }, { id: "INC-502", type: "warning", title: "Elevator Outage", desc: "Section 120 Elevator E-2 offline.", severity: "warning", time: "Now" }], alertsStatus: "1 CRITICAL EMERGENCY", alertsStatusClass: "bg-cyber-roseDim text-cyber-rose border-cyber-rose/40", mapState: { heatColors: { north: 'rgba(16, 185, 129, 0.2)', east: 'rgba(16, 185, 129, 0.2)', south: 'rgba(244, 63, 94, 0.4)', west: 'rgba(16, 185, 129, 0.2)', concourse: 'rgba(16, 185, 129, 0.2)', gateC: 'rgba(16, 185, 129, 0.3)', gateG: 'rgba(16, 185, 129, 0.3)' }, showAccessibilityRoute: true, showRainOverlay: false, warningNode: 'elevator120' },
```

```
chartData: [45, 55, 62, 58, 52, 48, 46, 45] } }; // --- RENDER DYNAMIC SVG STADIUM TWIN --- function renderStadiumSVG() { const state = scenarioData[currentScenario].mapState; const colors = state.heatColors; const svg = ` <svg viewBox="0 0 800 500" class="w-full h-full max-w-[650px] transition-all duration-500"> <!-- Background Gradients --> <radialGradient id="bgGlow" cx="50%" cy="50%" r="50%"> <stop offset="0%" stop-color="#00f0ff" stop-opacity="0.05" /> <stop offset="100%" stop-color="#09090b" stop-opacity="0" /> </radialGradient> <linearGradient id="cyberGreen" x1="0" y1="0" x2="1" y2="1"> <stop offset="0%" stop-color="#10b981" /> <stop offset="100%" stop-color="#059669" /> </linearGradient> <!-- Flow Arrow Path Pattern --> <pattern id="flowLines" width="20" height="20" patternUnits="userSpaceOnUse"> <line x1="0" y1="10" x2="20" y2="10" stroke="#00f0ff" stroke-width="2" stroke-dasharray="4,4" /> </pattern> </defs> <!-- Glow underlay --> <rect width="800" height="500" fill="url(#bgGlow)" /> <!-- Stadium Outer Ring / Parking & Transit Outer limits --> <circle cx="400" cy="230" r="220" fill="none" stroke="#27272a" stroke-width="2" stroke-dasharray="8,8" /> <!-- Parking / Transit Area Node --> <g id="transitHub" class="cursor-pointer group"> <circle cx="650" cy="120" r="24" fill="#18181b" stroke="${state.warningNode === 'transitHub' ? '#f43f5e' : '#27272a'}" stroke-width="2" class="transition-colors duration-500" /> <circle cx="650" cy="120" r="20" fill="rgba(0, 240, 255, 0.05)" /> <path d="M 642 120 L 658 120 M 650 112 L 650 128" stroke="#71717a" stroke-width="2" /> <text x="650" y="154" fill="#a1a1aa" font-size="10" font-family="JetBrains Mono" text-anchor="middle" font-weight="bold">METRO TERMINAL</text> ${state.warningNode === 'transitHub' ? '<circle cx="650" cy="120" r="30" fill="none" stroke="#f43f5e" stroke-width="1.5" class="animate-ping" />' : ''} </g> <!-- Parking Lot Vector -->
```

```
<g id="parkingLot" class="cursor-pointer"> <circle cx="150" cy="120" r="24" fill="#18181b" stroke="#27272a" stroke-width="2" /> <text x="150" y="124" fill="#71717a" font-size="12" font-family="JetBrains Mono" text-anchor="middle" font-weight="bold">P3</text> <text x="150" y="154" fill="#a1a1aa" font-size="10" font-family="JetBrains Mono" text-anchor="middle" font-weight="bold">SHUTTLE HUB</text> </g> <!-- Weather Overlay (Rains) --> ${state.showRainOverlay ? ` <g opacity="0.3"> <line x1="100" y1="20" x2="80" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" /> <line x1="250" y1="20" x2="230" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" /> <line x1="400" y1="20" x2="380" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" /> <line x1="550" y1="20" x2="530" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" /> <line x1="700" y2="20" x2="680" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" />` : ''}
```

```

y1="20" x2="680" y2="480" stroke="#00f0ff" stroke-width="1.5" stroke-dasharray="10,20" class="animate-rain" /> </g>
: '}' <!-- Concourse Outer Shell --> <ellipse cx="400" cy="230" rx="180" ry="140" fill="{colors.concourse}"
stroke="#3f3f46" stroke-width="3" class="transition-all duration-500" /> <!-- Stadium Seating Stands (4 Sectors) -->
<!-- North Stand --> <path d="M 280 130 C 350 90, 450 90, 520 130 L 490 160 C 440 130, 360 130, 310 160 Z"
fill="{colors.north}" stroke="{state.warningNode === 'sectorNorth' ? '#f43f5e' : '#3f3f46'}" stroke-width="2"
class="transition-all duration-500 cursor-pointer hover:opacity-80" /> <text x="400" y="115" fill="ffffff"
font-size="11" font-family="JetBrains Mono" text-anchor="middle" font-weight="bold">NORTH STAND</text> <!-- East
Stand --> <path d="M 540 150 C 600 200, 600 260, 540 310 L 510 280 C 550 250, 550 210, 510 180 Z"
fill="{colors.east}" stroke="#3f3f46" stroke-width="2" class="transition-all duration-500 cursor-pointer
hover:opacity-80" /> <text x="580" y="235" fill="ffffff" font-size="11" font-family="JetBrains Mono"
text-anchor="middle" font-weight="bold">EAST STAND</text> <!-- South Stand --> <path d="M 280 330 C 350 370, 450 370,
520 330 L 490 300 C 440 330, 360 330, 310 300 Z" fill="{colors.south}" stroke="#3f3f46" stroke-width="2"
class="transition-all duration-500 cursor-pointer hover:opacity-80" /> <text x="400" y="355" fill="ffffff"
font-size="11" font-family="JetBrains Mono" text-anchor="middle" font-weight="bold">SOUTH STAND</text> <!-- West
Stand --> <path d="M 260 150 C 200 200, 200 260, 260 310 L 290 280 C 250 250, 250 210, 290 180 Z"
fill="{colors.west}" stroke="#3f3f46" stroke-width="2" class="transition-all duration-500 cursor-pointer
hover:opacity-80" /> <text x="220" y="235" fill="ffffff" font-size="11" font-family="JetBrains Mono"
text-anchor="middle" font-weight="bold">WEST STAND</text> <!-- Concourse East Risk Node --> ${state.warningNode ===
'concourseEast' ? ` <g> <circle cx="530" cy="230" r="18" fill="rgba(244, 63, 94, 0.2)" stroke="#f43f5e"
stroke-width="2" class="animate-pulse" />

```

```

<circle cx="530" cy="230" r="8" fill="#f43f5e" /> <circle cx="530" cy="230" r="25" fill="none" stroke="#f43f5e"
stroke-width="1" class="animate-ping" /> </g> ` : '}' <!-- Pit Field (Green pitch) --> <rect x="330" y="180"
width="140" height="100" rx="6" fill="#14532d" stroke="#16a34a" stroke-width="2" /> <ellipse cx="400" cy="230"
rx="30" ry="20" fill="none" stroke="#16a34a" stroke-width="1.5" /> <line x1="400" y1="180" x2="400" y2="280"
stroke="#16a34a" stroke-width="1.5" /> <!-- GATES (Entry & Exit points) --> <!-- Gate A --> <circle cx="400" cy="70"
r="8" fill="#27272a" stroke="#71717a" stroke-width="2" /> <text x="400" y="58" fill="#a1a1aa" font-size="9"
font-family="JetBrains Mono" text-anchor="middle">GATE A</text> <!-- Gate C (Transit side) --> <g> <circle cx="590"
cy="180" r="8" fill="#27272a" stroke="{currentScenario === 'transit' ? '#f43f5e' : '#71717a'}" stroke-width="2"
class="transition-colors duration-500" /> <text x="615" y="183" fill="#a1a1aa" font-size="9" font-family="JetBrains
Mono">GATE C</text> </g> <!-- Gate G --> <circle cx="210" cy="280" r="8" fill="#27272a" stroke="#71717a"
stroke-width="2" /> <text x="185" y="284" fill="#a1a1aa" font-size="9" font-family="JetBrains Mono">GATE G</text>
<!-- Emergency Elevator Node --> <g id="elevator120" class="cursor-pointer"> <rect x="295" y="285" width="16"
height="16" rx="2" fill="#18181b" stroke="{state.warningNode === 'elevator120' ? '#f43f5e' : '#52525b'}"
stroke-width="2" class="transition-colors duration-500" /> <path d="M 300 293 L 303 290 L 306 293" fill="none"
stroke="#a1a1aa" stroke-width="1.5" /> <path d="M 300 295 L 303 298 L 306 295" fill="none" stroke="#a1a1aa"
stroke-width="1.5" /> ${state.warningNode === 'elevator120' ? ` <circle cx="303" cy="293" r="14" fill="none"
stroke="#f43f5e" stroke-width="1.5" class="animate-ping" /> ` : ''} </g> <!-- REROUTING PATHWAY / ACCESSIBILITY
VECTORS --> ${state.showAccessibilityRoute && activeInterventions.medical} ? ` <!-- DETOUR PATH (Dashed blue glowing
arrow) --> <g> <path id="detourPath" d="M 310 293 Q 320 310, 350 310 T 430 310 T 520 280" fill="none" stroke="#00f0ff"
stroke-width="4" stroke-linecap="round" stroke-dasharray="8,8" class="animate-flow" /> <text x="430" y="325"
fill="#00f0ff" font-size="9" font-family="JetBrains Mono" font-weight="bold" text-anchor="middle">ACTIVE Detour Path
(Dijkstra Bypass)</text> </g> ` : '}' <!-- Pedestrian Flow Vectors for Halftime / Transit scenario -->
${currentScenario === 'halftime' && activeInterventions.halftime} ? `

```

```

<g> <!-- Routing arrows north and south --> <path d="M 520 180 Q 480 160, 440 170" fill="none" stroke="#f59e0b"
stroke-width="3" stroke-linecap="round" stroke-dasharray="6,6" class="animate-flow" /> <path d="M 530 280 Q 480 300,
440 290" fill="none" stroke="#10b981" stroke-width="3" stroke-linecap="round" stroke-dasharray="6,6"
class="animate-flow" /> <text x="490" y="150" fill="#f59e0b" font-size="9" font-family="JetBrains Mono"
font-weight="bold" text-anchor="middle">Diverting 41% Vector North</text> </g> ` : '}' ${currentScenario ===
'transit' && activeInterventions.transit} ? ` <g> <!-- Transit hold arrows --> <path d="M 530 230 Q 560 210, 580 185"
fill="none" stroke="#f43f5e" stroke-width="3" stroke-linecap="round" stroke-dasharray="6,6" /> <!-- Cross barrier -->
<line x1="565" y1="180" x2="585" y2="200" stroke="#f43f5e" stroke-width="4" /> <line x1="585" y1="180" x2="565"
y2="200" stroke="#f43f5e" stroke-width="4" /> <text x="530" y="160" fill="#f43f5e" font-size="9"
font-family="JetBrains Mono" font-weight="bold" text-anchor="middle">GATE C PEDESTRIAN THRUPUT PAUSED</text> </g> ` :
'}` </svg> `; document.getElementById('mapContainer').innerHTML = svg; } // --- INITIALIZE ECHARTS LOAD PREDICTOR ---
let chartInstance = null; function renderChart() { const chartDom = document.getElementById('loadPredictorChart'); if
(!chartInstance) { chartInstance = echarts.init(chartDom); const option = { backgroundColor: 'transparent',
tooltip: { trigger: 'axis', backgroundColor: '#18181b', borderColor: '#27272a', textStyle: { color: '#fafafa',
fontFamily: 'JetBrains Mono, monospace', fontSize: 11 } },

```

```

grid: { top: '10%', left: '5%', right: '5%', bottom: '5%', containLabel: true }, xAxis: { type: 'category',
boundaryGap: false, data: ['T-15', 'T-10', 'T-5', 'Now', 'T+5', 'T+10', 'T+20', 'T+30'], axisLine: { lineStyle: {
color: '#3f3f46' } }, axisLabel: { color: '#71717a', fontSize: 10, fontFamily: 'JetBrains Mono' } }, yAxis: { type:
'value', splitLine: { lineStyle: { color: '#1e1e24' } }, axisLabel: { color: '#71717a', fontSize: 10, fontFamily:
'JetBrains Mono' } }, series: [{ name: 'Vector Density Load (Index)', data: scenarioData[currentScenario].chartData,
type: 'line', smooth: true, symbolSize: 6, lineStyle: { color: '#00f0ff', width: 2.5, shadowBlur: 8, shadowColor:
'rgba(0, 240, 255, 0.4)' }, itemStyle: { color: '#00f0ff' }, areaStyle: { color: new
echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: 'rgba(0, 240, 255, 0.25)' }, { offset: 1, color:
'rgba(0, 240, 255, 0)' }]) } }] }]; chartInstance.setOption(option); }

```

```
// --- LIVE GEMINI API INTEGRATION --- const liveNarrativeCache = {}; // Setup API Key Input logic
document.addEventListener("DOMContentLoaded", () => { const keyInput = document.getElementById('geminiApiKey'); if
(keyInput) { // Load cached key const savedKey = localStorage.getItem('gemini_api_key'); if (savedKey) {
keyInput.value = savedKey; } // Listen for key changes keyInput.addEventListener('input', (e) => {
localStorage.setItem('gemini_api_key', e.target.value.trim()); // Clear cache when key changes to force refresh for
(let k in liveNarrativeCache) delete liveNarrativeCache[k]; updateDashboard(); }); }); async function
fetchLiveGeminiNarrative(scenario, step) { const keyInput = document.getElementById('geminiApiKey'); const apiKey =
keyInput ? keyInput.value.trim() : ''; if (!apiKey) return null; const cacheKey = `${scenario}_${step}`; if
(liveNarrativeCache[cacheKey]) { return liveNarrativeCache[cacheKey]; } const data = scenarioData[scenario]; const
timeLabel = ["Reality (T+0)", "T+5m (Imagined Future)", "T+10m (Critical Future)", "T+30m (Mitigated Future)"][step];
const prompt = `You are the Cognitive Core Brain of Stadium Pulse running in the stadium operations room. Analyze the
current stadium telemetry for the scenario "${data.label}" at time horizon "${timeLabel}". Current crowd density:
${data.metrics.density[step]} Current flow rate: ${data.metrics.flow[step]} Active Incidents:
${JSON.stringify(data.incidents)} Sub-agent negotiation state: ${JSON.stringify(data.negotiations)} Generate a highly
realistic, futuristic operational summary of 2-3 sentences.`
```

```
1. Describe the situation or predicted outcome. 2. Formulate a direct command for volunteer and emergency agents to
resolve or stabilize it. Use bolding on critical zones or actions. Start with ***[Live Gemini Core]***.`; try {
const response = await
fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${apiKey}`, {
method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ contents: [{ parts: [{ text:
prompt }] }] }); if (!response.ok) { const errData = await response.json(); throw new Error(errData.error?.message
|| response.statusText); } const resData = await response.json(); const genText =
resData.candidates?.[0]?.content?.parts?.[0]?.text || "No response generated."; liveNarrativeCache[cacheKey] =
genText.trim(); return liveNarrativeCache[cacheKey]; } catch (e) { console.error("Gemini API Error:", e); return
`***[Live Core Error]*** Failed to query Gemini API: ${e.message}`; } } // --- UPDATE UI PANELS ON STATE CHANGE ---
function updateDashboard() { const data = scenarioData[currentScenario]; // Update Header active label
document.getElementById('activeScenarioLabel').textContent = data.label; // Update KPIs
document.getElementById('kpiDensity').innerHTML = `${data.metrics.density[currentTimelineStep]}<span class="text-sm
font-normal text-zinc-400">/m²`; // Update KPI 1 Trend badge const densityTrend =
document.getElementById('kpiDensityTrend'); densityTrend.textContent =
getDensityTrendText(data.metrics.density[currentTimelineStep]); densityTrend.className = `text-xs px-2 py-0.5
rounded-full font-mono font-semibold ${data.metrics.densityClass[currentTimelineStep]}`;
```

```
// Update KPI 2 document.getElementById('kpiFlow').textContent = data.metrics.flow[currentTimelineStep]; const
flowTrend = document.getElementById('kpiFlowTrend'); flowTrend.textContent =
getFlowTrendText(data.metrics.flow[currentTimelineStep]); flowTrend.className = `text-xs px-2 py-0.5 rounded-full
font-mono font-semibold ${data.metrics.flowClass[currentTimelineStep]}`; // Update KPI 3
document.getElementById('kpiEnergy').textContent = data.metrics.energy[currentTimelineStep]; const energyTrend =
document.getElementById('kpiEnergyTrend'); energyTrend.textContent = data.metrics.energySaving[currentTimelineStep];
// Update KPI 4 document.getElementById('kpiVolunteers').textContent = data.metrics.volunteers[currentTimelineStep];
const volTrend = document.getElementById('kpiVolunteersTrend'); volTrend.className = `text-xs px-2 py-0.5
rounded-full font-mono font-semibold ${data.metrics.volunteersClass[currentTimelineStep]}`; // Update Storyteller
Narrator (using formatted markdown/HTML) const narratorEl = document.getElementById('storytellerNarrator');
narratorEl.innerHTML = ''; const keyInput = document.getElementById('geminiApiKey'); const hasKey = keyInput &&
keyInput.value.trim().length > 0; if (!hasKey) { // Render Tip const tip = document.createElement('div');
tip.className = 'text-[11px] bg-cyber-cyanDim border border-cyber-cyan/30 text-cyber-cyan rounded p-2 mb-3 font-mono
leading-snug'; tip.innerHTML = '■ LIVE AI MODE AVAILABLE
Enter a Google Gemini API Key in the
header to run live Generative AI analysis of stadium events!'; narratorEl.appendChild(tip); // Render all narrative
items up to current timeline step for (let i = 0; i <= currentTimelineStep; i++) { const p =
document.createElement('p'); p.className = i === currentTimelineStep ? 'text-cyber-cyan font-semibold border-l-2
border-cyber-cyan pl-2 py-1' : 'text-zinc-400'; p.innerHTML = formatMarkdown(data.narrator[i]);
narratorEl.appendChild(p); } else { // Render Active Status const status = document.createElement('div');
status.className = 'text-[11px] bg-cyber-emeraldDim border border-cyber-emerald/30 text-cyber-emerald rounded p-2
mb-3 font-mono leading-snug flex items-center gap-2'; status.innerHTML = '<span class="w-1.5 h-1.5 rounded-full
bg-cyber-emerald animate-ping">LIVE GENERATIVE CORE ACTIVE';
narratorEl.appendChild(status); for (let i = 0; i <= currentTimelineStep; i++) {
```

```
const p = document.createElement('p'); p.className = i === currentTimelineStep ? 'text-cyber-cyan font-semibold
border-l-2 border-cyber-cyan pl-2 py-1' : 'text-zinc-400'; const cacheKey = `${currentScenario}_${i}`; if
(liveNarrativeCache[cacheKey]) { p.innerHTML = formatMarkdown(liveNarrativeCache[cacheKey]); } else { p.innerHTML =
`<em>***[Live Gemini Core]*** Thinking...</em>`; // Trigger async fetch fetchLiveGeminiNarrative(currentScenario,
i).then(text => { updateDashboard(); }); } narratorEl.appendChild(p); } } narratorEl.scrollTop =
narratorEl.scrollHeight; // Update Agent Negotiation logs const negotiationEl =
document.getElementById('agentNegotiationFeed'); negotiationEl.innerHTML = ''; data.negotiations.forEach(log => {
const div = document.createElement('div'); div.className = 'border-b border-zinc-800 pb-1.5 hover:bg-zinc-800/40 p-1
rounded transition-colors'; div.innerHTML = colorizeLog(log); negotiationEl.appendChild(div); });
negotiationEl.scrollTop = negotiationEl.scrollHeight; // Update Incident panel const incidentEl =
document.getElementById('incidentLogs'); incidentEl.innerHTML = ''; const alertCountBadge =
document.getElementById('alertCount'); alertCountBadge.textContent = data.alertsStatus; alertCountBadge.className =
`text-xs font-mono border px-2 py-0.5 rounded font-bold uppercase ${data.alertsStatusClass}`;
```

```
data.incidents.forEach(inc => { const item = document.createElement('div'); let sevClass = ''; let icon = 'info'; if (inc.severity === 'danger') { sevClass = 'border-cyber-rose/30 bg-cyber-roseDim text-cyber-rose'; icon = 'shield-alert';
```

```
} else if (inc.severity === 'warning') { sevClass = 'border-cyber-amber/30 bg-cyber-amberDim text-cyber-amber'; icon = 'alert-triangle'; } else { sevClass = 'border-zinc-800 bg-zinc-900/60 text-zinc-300'; icon = 'check-circle'; } item.className = 'border rounded-lg p-3 ${sevClass} flex gap-3 text-xs'; item.innerHTML = `<div class="mt-0.5"><i data-lucide="${icon}" class="w-4 h-4"></i></div> <div class="flex-1"> <div class="font-bold flex justify-between"> <span>${inc.title}</span> <span class="text-[10px] opacity-60 font-mono">${inc.time}</span> </div> <p class="opacity-80 mt-1 font-mono">${inc.desc}</p> </div> `; incidentEl.appendChild(item); }); // Update Intervention Card UI const interventionTextEl = document.getElementById('interventionText'); const btnExecuteEl = document.getElementById('btnExecuteIntervention'); const btnExecuteTextEl = document.getElementById('btnExecuteText'); if (currentScenario === 'base') { interventionTextEl.textContent = interventionsData.base.text; btnExecuteEl.disabled = true; btnExecuteEl.className = "w-full bg-zinc-800 border border-zinc-700 text-zinc-500 font-bold uppercase text-[10px] py-2 px-3 rounded-lg cursor-not-allowed transition-all flex items-center justify-center gap-1.5"; btnExecuteTextEl.textContent = interventionsData.base.action; } else { interventionTextEl.textContent = interventionsData[currentScenario].text; if (!activeInterventions[currentScenario]) { btnExecuteEl.disabled = false; btnExecuteEl.className = "w-full bg-cyber-cyanDim hover:bg-cyber-cyan/20 border border-cyber-cyan text-cyber-cyan font-bold uppercase text-[10px] py-2 px-3 rounded-lg cursor-pointer transition-all flex items-center justify-center gap-1.5 glow-box"; btnExecuteTextEl.textContent = "Execute: " + interventionsData[currentScenario].action; } else { btnExecuteEl.disabled = true; btnExecuteEl.className = "w-full bg-cyber-emeraldDim border border-cyber-emerald text-cyber-emerald font-bold uppercase text-[10px] py-2 px-3 rounded-lg cursor-default transition-all flex items-center justify-center gap-1.5"; btnExecuteTextEl.textContent = "Mitigation Active"; } } lucide.createIcons();
```

```
// Render Stadium Map renderStadiumSVG(); // Render charts renderChart(); } // Helpers to dynamically resolve status based on metrics function getDensityTrendText(densityStr) { const val = parseFloat(densityStr); if (val > 3.5) return 'SEVERE'; if (val > 2.5) return 'WARNING'; if (val > 1.5) return 'HEAVY'; return 'OPTIMAL'; } function getFlowTrendText(flowStr) { const val = parseFloat(flowStr); if (val < 70) return 'BLOCKED'; if (val < 85) return 'RESTRICTED'; return 'STABLE'; } // Secure HTML Escaper function escapeHTML(str) { return str.replace(/[<>"]/g, tag => ({ '&': '&', '<': '<', '>': '>', '"': '"', "'": ''' })[tag] || tag); } // Markdown Formatter (Bold parsing) with XSS Protection function formatMarkdown(text) { const escaped = escapeHTML(text); return escaped.replace(/^(.*)?(.*)$/g, '<strong class="text-white font-extrabold">$1'); } // Colorize Logs for futuristic display function colorizeLog(log) {
```

```
return log .replace(/\[System\]/g, '[System]') .replace(/\[Weather Agent\]/g, '[Weather Agent]') .replace(/\[Energy Agent\]/g, '[Energy Agent]') .replace(/\[Crowd Agent\]/g, '[Crowd Agent]') .replace(/\[Accessibility Agent\]/g, '[Accessibility Agent]') .replace(/\[Volunteer Agent\]/g, '[Volunteer Agent]') .replace(/\[Transport Agent\]/g, '[Transport Agent]') .replace(/\[Medical Agent\]/g, '[Medical Agent]') .replace(/\[Waste Agent\]/g, '[Waste Agent]') .replace(/-/g, '-'); } // --- INTERACTIVE EVENT LISTENERS --- // Execute Intervention Button const btnExecuteEl = document.getElementById('btnExecuteIntervention'); if (btnExecuteEl) { btnExecuteEl.addEventListener('click', () => { if (currentScenario !== 'base' && !activeInterventions[currentScenario]) { activeInterventions[currentScenario] = true; // Push system negotiation logs const logMsg = `[System] -> [All Agents]: Operator approved Gemini Core mitigation. Deploying routing vectors...`; scenarioData[currentScenario].negotiations.push(logMsg); // Shift timeline state to T+30m: MITIGATED STATE automatically currentTimelineStep = 3; document.getElementById('timelineRange').value = 3; updateTimelineLabel(); updateDashboard(); } }); } // Scenario Buttons document.querySelectorAll('.scenario-btn').forEach(btn => { btn.addEventListener('click', (e) => { const targetBtn = e.currentTarget; document.querySelectorAll('.scenario-btn').forEach(b => { b.className = 'scenario-btn flex items-center justify-center gap-2 py-2 px-3 rounded-lg text-xs font-mono font-bold uppercase tracking-wider border border-zinc-800 bg-zinc-900 text-zinc-400 hover:text-white hover:border-zinc-700 transition-all'; }); targetBtn.className = 'scenario-btn active-btn flex items-center justify-center gap-2 py-2 px-3 rounded-lg text-xs font-mono font-bold uppercase tracking-wider border transition-all'; currentScenario = targetBtn.dataset.scenario;
```

```
// Reset slider to Reality when changing scenario currentTimelineStep = 0; document.getElementById('timelineRange').value = 0; updateTimelineLabel(); updateDashboard(); }); }); // Timeline Slider const timelineRange = document.getElementById('timelineRange'); timelineRange.addEventListener('input', (e) => { currentTimelineStep = parseInt(e.target.value); updateTimelineLabel(); updateDashboard(); }); // Timeline Markers document.querySelectorAll('.timeline-marker').forEach(marker => { marker.addEventListener('click', (e) => { const step = parseInt(e.currentTarget.dataset.step); currentTimelineStep = step; timelineRange.value = step; updateTimelineLabel(); updateDashboard(); }); }); function updateTimelineLabel() { const labels = ["T+0: REALITY STATE", "T+5m: IMAGINED FUTURE", "T+10m: CRITICAL FUTURE", "T+30m: MITIGATED STATE"]; document.getElementById('timelineLabel').textContent = labels[currentTimelineStep]; // Highlight active timeline marker text document.querySelectorAll('.timeline-marker').forEach((marker, idx) => { if (idx === currentTimelineStep) { marker.classList.add('active-marker'); marker.classList.remove('text-zinc-500'); marker.classList.add('text-cyber-cyan'); } else { marker.classList.remove('active-marker'); marker.classList.add('text-zinc-500'); marker.classList.remove('text-cyber-cyan'); } });
```



```

} // --- NEURAL NETWORK CANVAS ANIMATION --- const canvas = document.getElementById('neuralCanvas'); const ctx =
canvas.getContext('2d'); let points = []; const maxPoints = 50; function resizeCanvas() { canvas.width =
window.innerWidth; canvas.height = window.innerHeight; initPoints(); } function initPoints() { points = []; for (let
i = 0; i < maxPoints; i++) { points.push({ x: Math.random() * canvas.width, y: Math.random() * canvas.height, vx:
(Math.random() - 0.5) * 0.5, vy: (Math.random() - 0.5) * 0.5, radius: Math.random() * 2 + 1 }); } } function
drawNeuralNetwork() { // Performance Optimization: Halt rendering if tab is hidden/minimized if (document.hidden) {
requestAnimationFrame(drawNeuralNetwork); return; } ctx.clearRect(0, 0, canvas.width, canvas.height); ctx.fillStyle =
'#09090b'; ctx.fillRect(0, 0, canvas.width, canvas.height); ctx.fillStyle = 'rgba(0, 240, 255, 0.4)'; ctx.strokeStyle =
'rgba(0, 240, 255, 0.05)'; for (let i = 0; i < points.length; i++) { let p = points[i];

// Move points p.x += p.vx; p.y += p.vy; // Boundaries if (p.x < 0 || p.x > canvas.width) p.vx *= -1; if (p.y < 0 ||
p.y > canvas.height) p.vy *= -1; // Draw node ctx.beginPath(); ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
ctx.fill(); // Connect nodes for (let j = i + 1; j < points.length; j++) { let p2 = points[j]; let dist =
Math.hypot(p.x - p2.x, p.y - p2.y); if (dist < 120) { ctx.lineWidth = 1 - dist / 120; ctx.beginPath(); ctx.moveTo(p.x,
p.y); ctx.lineTo(p2.x, p2.y); ctx.stroke(); } } } requestAnimationFrame(drawNeuralNetwork); }
window.addEventListener('resize', resizeCanvas); resizeCanvas(); drawNeuralNetwork(); // Initialize Dashboard
Rendering window.onload = () => { updateDashboard(); }; window.addEventListener('resize', () => { if (chartInstance)
{ chartInstance.resize(); } });

```

### 3.3 Glassmorphic Styling and Animations

```

/* STADIUM PULSE - FUTURISTIC DESIGN SYSTEM & GLOWING EFFECTS */ /* Custom Animations */ @keyframes rainFlow { 0% {
stroke-dashoffset: 0; } 100% { stroke-dashoffset: 200; } } @keyframes dashFlow { 0% { stroke-dashoffset: 160; } 100% {
stroke-dashoffset: 0; } } .animate-rain { animation: rainFlow 1.5s linear infinite; } .animate-flow { animation:
dashFlow 3s linear infinite; } /* Glassmorphic Cyber Cards */ .bg-cyber-card { background: rgba(10, 10, 14, 0.85);
backdrop-filter: blur(12px); border: 1px solid rgba(255, 255, 255, 0.08); box-shadow: 0 8px 32px 0 rgba(0, 0, 0,
0.37); transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1); } .bg-cyber-card:hover { border-color: rgba(0, 240, 255,
0.25); box-shadow: 0 8px 32px 0 rgba(0, 240, 255, 0.05); } /* Glowing Boxes */ .glow-box {

box-shadow: 0 0 15px 0 rgba(0, 240, 255, 0.03); border: 1px solid rgba(0, 240, 255, 0.15) !important; } /* Active
Scenario Buttons */ .active-btn { background: rgba(0, 240, 255, 0.1) !important; border-color: #00f0ff !important;
color: #00f0ff !important; box-shadow: 0 0 12px 0 rgba(0, 240, 255, 0.25); text-shadow: 0 0 8px rgba(0, 240, 255,
0.5); } /* Timeline Slider Styles */ #timelineRange::-webkit-slider-thumb { width: 16px; height: 16px; border-radius:
50%; background: #00f0ff; cursor: pointer; box-shadow: 0 0 10px 0 #00f0ff; transition: transform 0.1s; }
#timelineRange::-webkit-slider-thumb:hover { transform: scale(1.2); } /* Scrollbar Customization */
.scrollbar-custom::-webkit-scrollbar { width: 4px; } .scrollbar-custom::-webkit-scrollbar-track { background:
transparent; } .scrollbar-custom::-webkit-scrollbar-thumb { background: rgba(255, 255, 255, 0.1); border-radius: 2px; }
.scrollbar-custom::-webkit-scrollbar-thumb:hover { background: rgba(0, 240, 255, 0.4); }

/* Map Interactive Nodes hover states */ .cursor-pointer:hover { filter: drop-shadow(0 0 4px rgba(0, 240, 255, 0.3)); }
/* Active timeline marker */ .active-marker { text-shadow: 0 0 6px rgba(0, 240, 255, 0.6); }

```

### 3.4 Automated Testing & Validation Suite

```

import unittest import math import os import re class TestStadiumPulseSystem(unittest.TestCase): def setUp(self): #
Setup mock spatial graph topology for Dijkstra test self.graph = { 'V1': { 'V2': 5, 'V3': 10}, 'V2': { 'V5': 15}, 'V3':
{ 'V4': 2, 'V5': 8}, 'V4': { 'V5': 3}, 'V5': {} } def dijkstra(self, start, end, blocked_nodes=None): if blocked_nodes
is None: blocked_nodes = set() distances = {node: float('inf') for node in self.graph} distances[start] = 0 visited =
set() while len(visited) < len(self.graph): curr = None curr_dist = float('inf') for n in self.graph: if n not in
visited and distances[n] < curr_dist: curr = n curr_dist = distances[n] if curr is None or curr == end: break
visited.add(curr) for neighbor, weight in self.graph[curr].items(): if neighbor in blocked_nodes: continue new_dist =
distances[curr] + weight if new_dist < distances[neighbor]: distances[neighbor] = new_dist return distances[end]

def test_accessibility_routing_dijkstra(self): """Verify Dijkstra calculations detour wheelchair paths when elevator
nodes are blocked.""" normal_path_cost = self.dijkstra('V1', 'V5') self.assertEqual(normal_path_cost, 15)
blocked_path_cost = self.dijkstra('V1', 'V5', blocked_nodes={'V3'}) self.assertEqual(blocked_path_cost, 20) def
test_social_force_crowd_dynamics(self): """Test physical crowd interaction force calculation formulas.""" r_i, r_j =
0.3, 0.3 # Body radius distance_normal = 1.0 distance_crush = 0.4 B = 0.08 force_normal = math.exp(((r_i + r_j) -
distance_normal) / B) force_crush = math.exp(((r_i + r_j) - distance_crush) / B) self.assertGreater(force_crush,
force_normal) self.assertGreater(force_crush, 1.0) def test_spatial_hashing_bounds(self): """Validate geohashing cell
indexing bounds.""" def get_h3_cell(x, y, cell_size=3.0): return int(x // cell_size), int(y // cell_size) cell1 =
get_h3_cell(1.5, 2.5) cell2 = get_h3_cell(1.8, 2.9) cell3 = get_h3_cell(4.2, 5.1) self.assertEqual(cell1, cell2)
self.assertNotEqual(cell1, cell3) def test_html_structure_integrity(self): """Parse index.html and verify that all
key functional DOM elements are present.""" self.assertTrue(os.path.exists("index.html"), "index.html does not
exist") with open("index.html", "r", encoding="utf-8") as f: html = f.read() # Check for crucial container IDs

```

```

required_ids = ["mapContainer", "storytellerNarrator", "agentNegotiationFeed",

"loadPredictorChart", "incidentLogs", "activeScenarioLabel", "timelineRange", "geminiApiKey",
"btnExecuteIntervention", "interventionText", "btnExecuteText"] for dom_id in required_ids:
self.assertIn(f'id="{dom_id}"', html, f'index.html is missing required element ID: {dom_id}') def
test_accessibility_compliance_tags(self): """Verify index.html contains correct accessibility tags (ARIA labels)."""
with open("index.html", "r", encoding="utf-8") as f: html = f.read() # Verify aria-labels are present on range input
and buttons self.assertIn('aria-label="Predictive time horizon slider"', html) self.assertIn('aria-label="Google
Gemini API Key for live AI reasoning"', html) self.assertIn('aria-label="Execute dynamic AI intervention"', html) #
Check explicit label relationships (for attributes) self.assertIn('for="layerCrowd"', html)
self.assertIn('for="layerAccessibility"', html) self.assertIn('for="layerVolunteers"', html)
self.assertIn('for="layerTransit"', html) def test_security_xss_protection_in_js(self): """Verify that app.js
implements character escaping (XSS protection) for AI text.""" self.assertTrue(os.path.exists("app.js"), "app.js does
not exist") with open("app.js", "r", encoding="utf-8") as f: js = f.read() # Check that XSS escape helper exists and
is used in formatMarkdown self.assertIn("function escapeHTML", js, "app.js is missing escapeHTML XSS sanitizer")
self.assertIn("escapeHTML(text)", js, "app.js does not apply escapeHTML sanitization in formatMarkdown") # Check
against dangerous code injections self.assertNotIn("eval(", js, "app.js contains unsafe eval() executions")
self.assertNotIn("document.write(", js, "app.js contains unsafe document.write() statements") if __name__ ==
 "__main__": unittest.main()

```

### 3.5 Containerized Deployment Config

```

FROM python:3.9-slim # Prevent python from writing pyc files and buffering stdout/stderr ENV
PYTHONDONTWRITEBYTECODE=1 ENV PYTHONUNBUFFERED=1 WORKDIR /app # Copy all project files COPY . . # Expose port EXPOSE
8080 # Run the python server script CMD ["python", "run.py"]

```

### 3.6 Local Web Server startup script

```

import http.server import socketserver import webbrowser import threading import time import sys import os PORT =
int(os.environ.get("PORT", 8000)) IS_CONTAINER = os.environ.get("K_SERVICE") is not None DIRECTORY = "." class
Handler(http.server.SimpleHTTPRequestHandler): def __init__(self, *args, **kwargs): super().__init__(*args, **kwargs)
def start_server(): # Allow quick reuse of the port socketserver.TCPServer.allow_reuse_address = True with
socketserver.TCPServer(("", PORT), Handler) as httpd: print(f"\n[STADIUM PULSE] Server successfully started at
http://localhost:{PORT}") print("[STADIUM PULSE] Press Ctrl+C to terminate the simulator.") try:
httpd.serve_forever() except KeyboardInterrupt: print("\n[STADIUM PULSE] Server terminated.") sys.exit(0) def
open_browser(): # Wait 1 second for server to initialize time.sleep(1.0) url = f"http://localhost:{PORT}/index.html"
print(f"[STADIUM PULSE] Launching simulator interface: {url}") webbrowser.open(url) if __name__ == "__main__":
print("=" * 60) print("STADIUM PULSE – THE LIVING COGNITIVE STADIUM COHESION SYSTEM") print("=" * 60) print("[STADIUM
PULSE] Initializing neural model simulation Twin...") # Run browser launcher in separate thread if not in container
if not IS_CONTAINER: threading.Thread(target=open_browser, daemon=True).start()

Run HTTP Server start_server()

```